Social Nest

Social Media Feed System

Session 2024 – 2028

Submitted by:
Muhammad Furqan Mustafa 2024-CS-100

Supervised by:
Miss Rabeeya Saleem

Course:
DSA (Data Structure & Algorithm)

Department of Computer Science
University of Engineering and Technology
Lahore Pakistan

Table of Contents

Social Nest ***1***

Objectives: **3**

Data Structures & Algorithms Used: **3**

1) Singly Linked List: 3
2) Queue (FIFO): 3
3) Stack (LIFO): 3
4) Insertion Sort: 4
5) Quick Sort: 4
6) Linear Search: 4

System Flow Diagram: **4**

Implementation Details: **5**

Screenshots of Working Output: **6**

Performance Analysis and Time Complexity: **11**

Conclusion: **11**

Objectives:

- To implement real-world social media features using fundamental and advanced Data Structures & Algorithms
- To demonstrate practical use of **Linked Lists, Doubly Linked Lists, Queues, Stacks, Hash Maps, Insertion Sort, Quick Sort, Linear Search**
- To build a fully functional, persistent (file-based), and beautiful GUI application using **Tkinter**
- To simulate **LinkedIn-like behavior** with feed, posts, friends, likes, comments, search, and undo functionality

Data Structures & Algorithms Used:

1) Singly Linked List:

- Storing all registered users
- Storing each user's personal friend list
- Dynamic size (no fixed limit)
- Easy insertion/deletion of users and friends
- Natural ordering of registration/friend addition

2) Queue (FIFO):

- Global post feed (implemented via **Doubly Linked List**)
- Comment ordering inside each post (Python list used as FIFO queue)
- Newest posts appear at the top
- First comment posted appears first — exactly like real social media
- O(1) insertion of new post at the beginning (head)
- O(1) deletion of any post
- Efficient forward traversal from newest to oldest
- Pending friend requests

3) Stack (LIFO):

- Navigation history (Back button)
- Undo functionality (undo delete post, undo navigation)

- Perfect for “go back” and “undo last action” features

4) Insertion Sort:

- Sorting posts by number of likes (Trending view)
- Very efficient when data is nearly sorted (few new likes at a time)
- Stable and adaptive

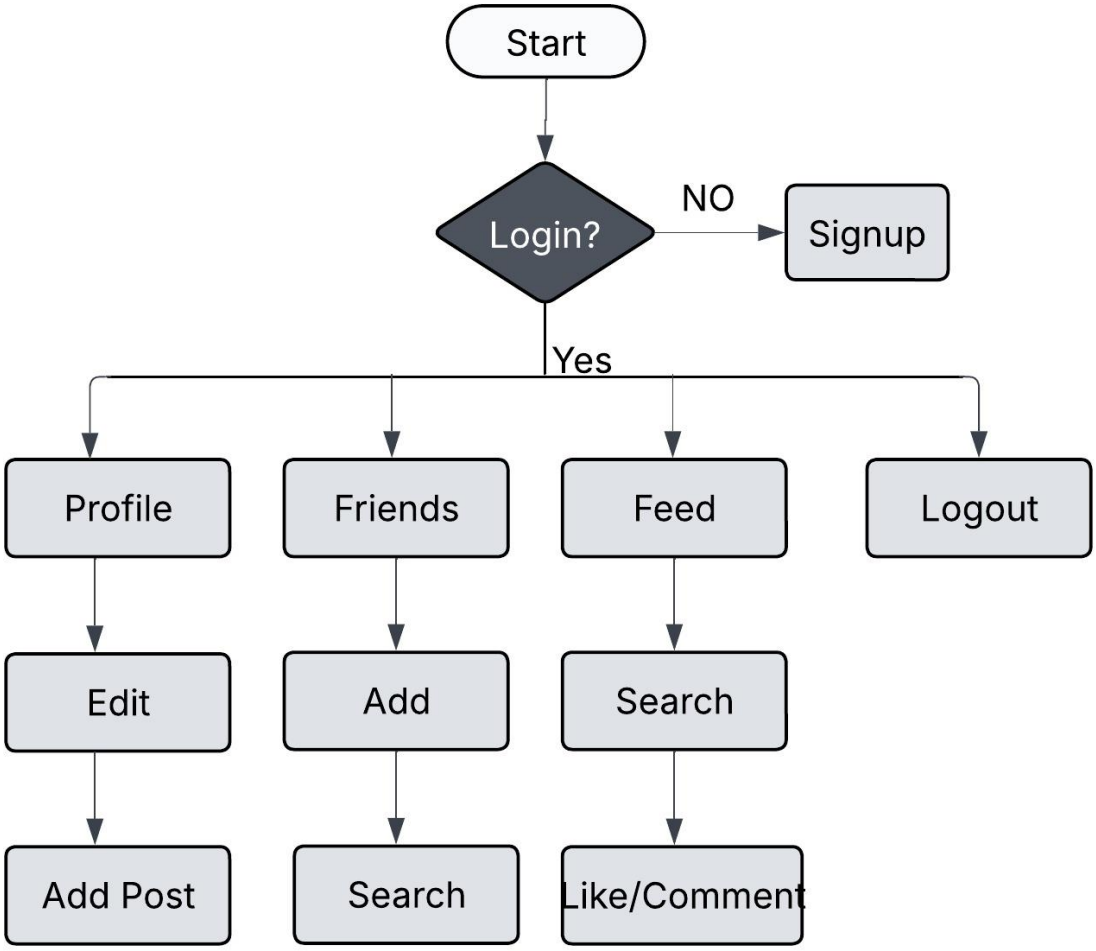
5) Quick Sort:

- Alphabetical sorting of friends and search results
- Average O(n log n) — one of the fastest comparison-based sorts
- In-place sorting

6) Linear Search:

- Keyword search in posts and user names
- Simple and effective on unsorted or partially sorted text data

System Flow Diagram:



Implementation Details:

Module 1: User Management (users.json)

- Users stored in **Singly Linked List**
- Fast lookup using **email → node** Hash Map
- Supports: Register, Login, Edit Profile
- Search by name using **Linear Search**

Module 2: Post Management (posts.json)

- Posts stored in **Doubly Linked List** acting as **Queue**
- New post → Insert at head → **O(1)**
- Like/Comment → **O(1)** using post_map
- Delete post → Remove node using prev/next pointers
- Comment system uses **FIFO Queue** (list)

Module 3: Feed Management (Core DSA Logic)

- Feed displays posts from **head → tail** → newest first • Supports:
  - View feed (FIFO)
    - Like/Unlike
    - Comment
    - Delete post
    - **Insertion Sort** for "Most Liked" view
    - **Quick Sort** for "A-Z" sorting

Module 4: Browsing History & Undo System

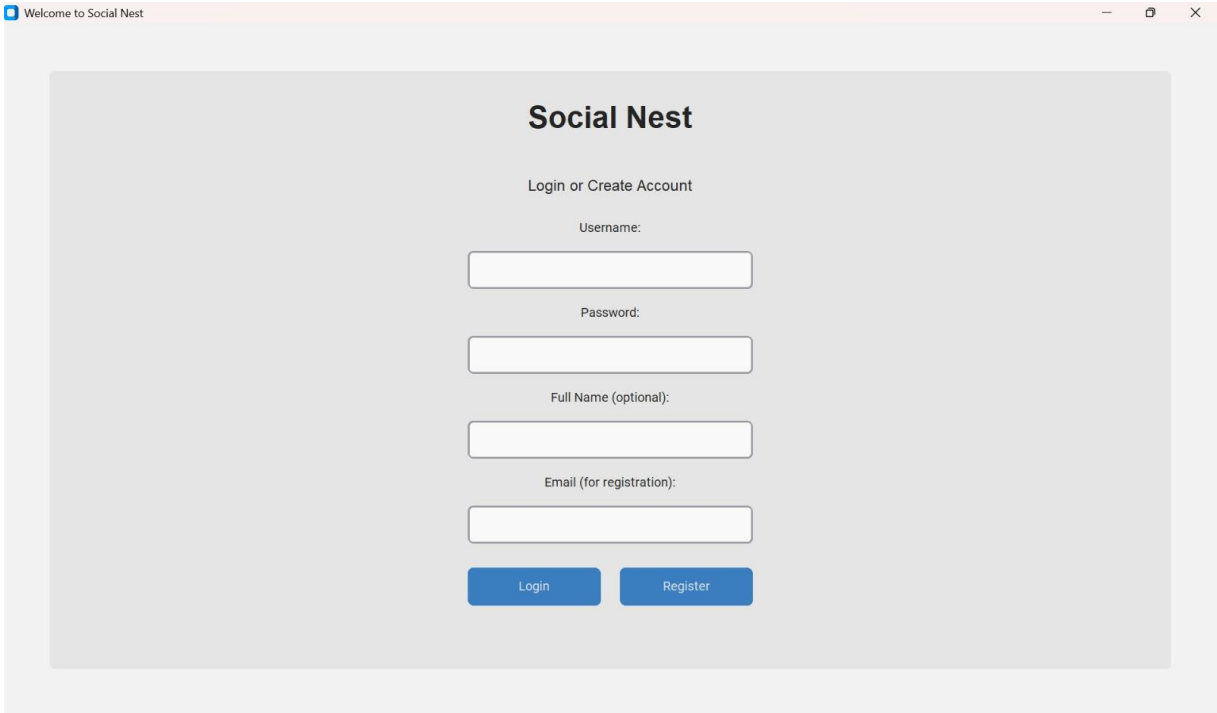
- Navigation tracked using **Stack**
- Back button = stack.pop()
- Supports **Undo Delete Post** (store deleted post in stack)

Module 5: Search & Sorting Module

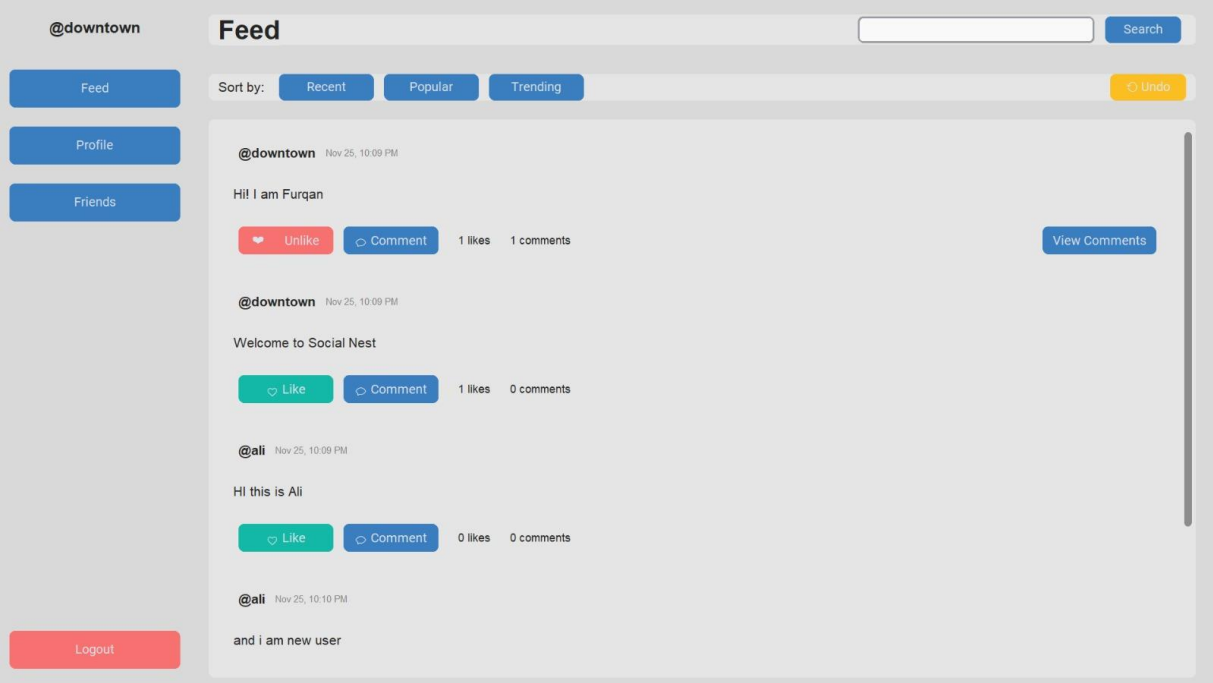
- Search posts by keyword → Linear search
- Search friends by name → Linear search on linked list
- Sort friends alphabetically → **Quick Sort**
- Sort posts by popularity → **Insertion Sort**

Screenshots of Working Output:

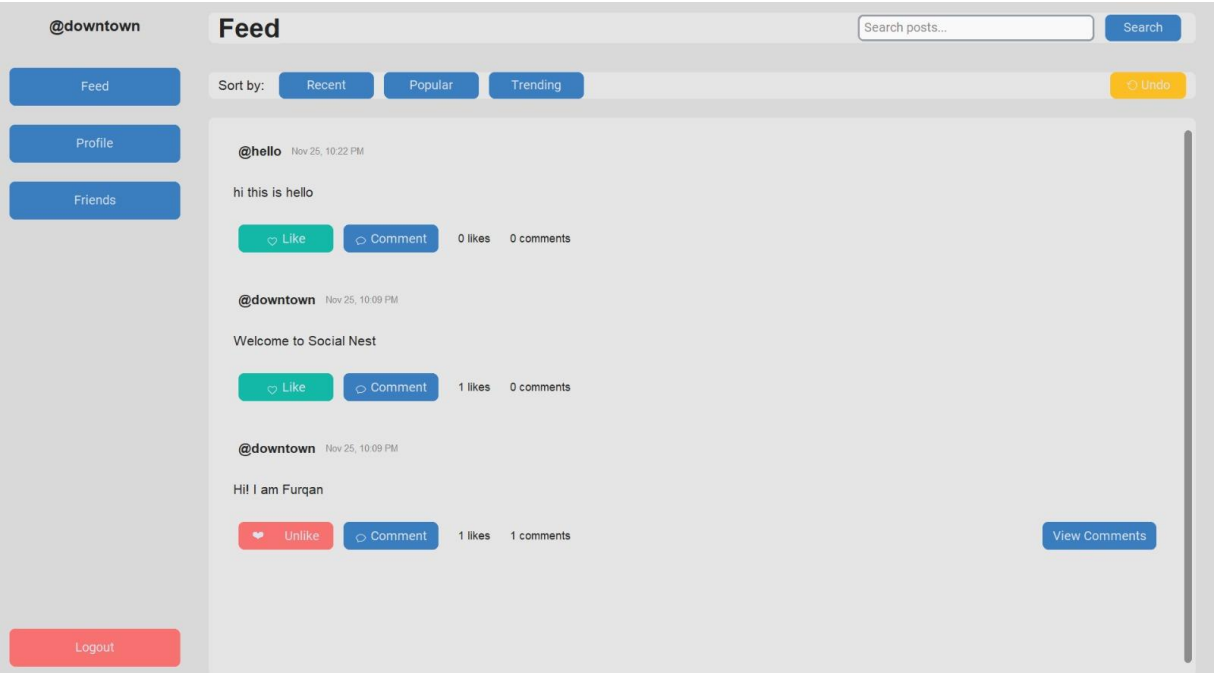
1) Beautiful Light Login Page:



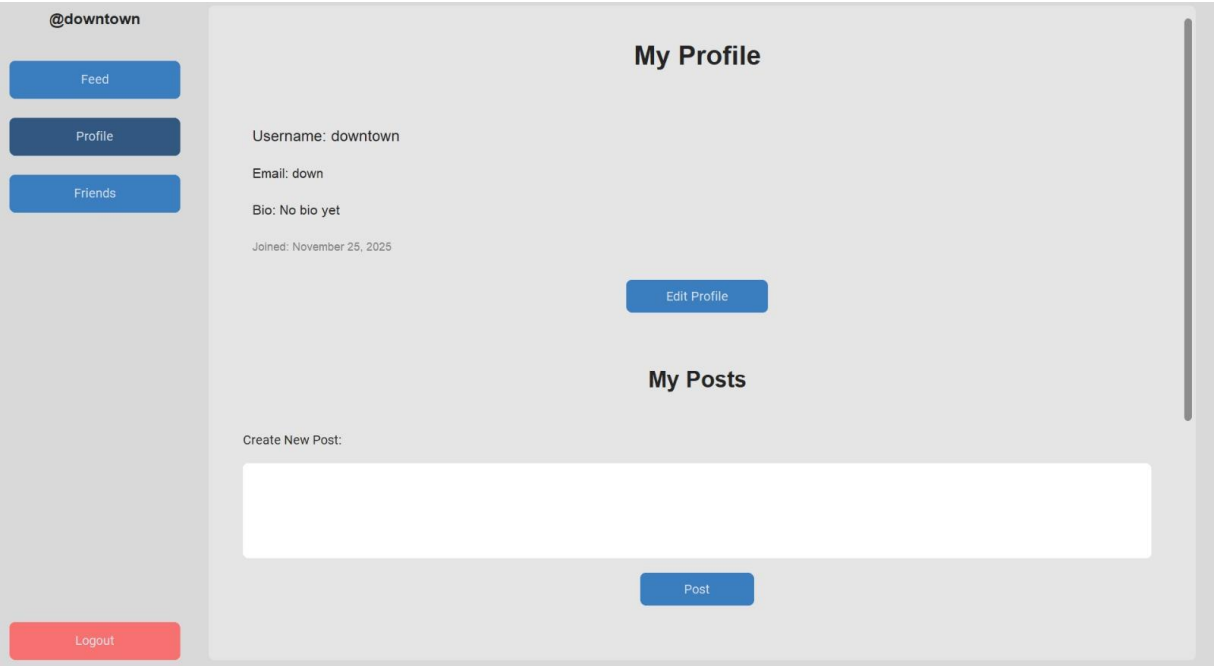
2) Home Page with Side bar and Navigation Stack:



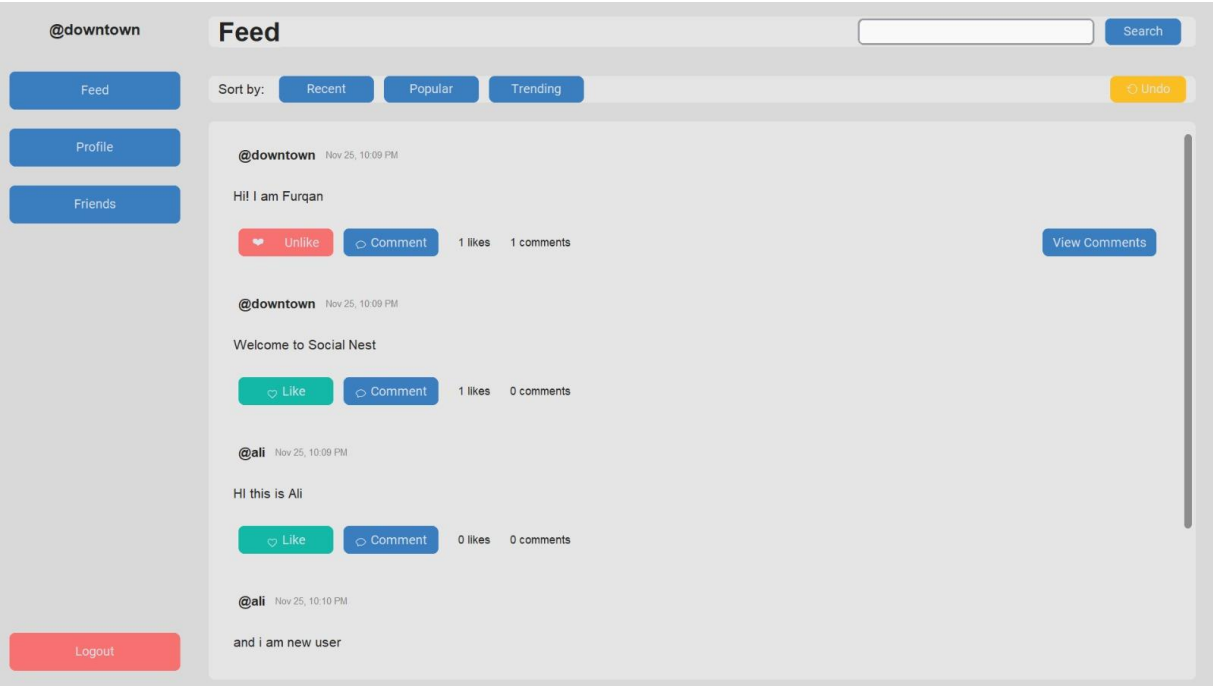
3) Feed Page with Scrollable Post:



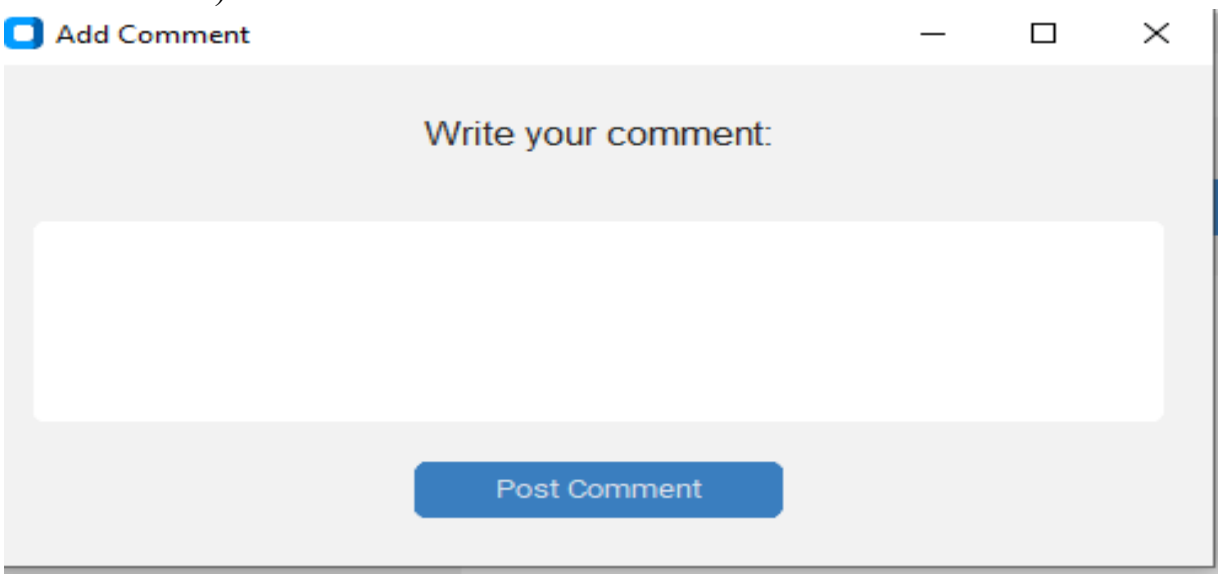
4) Post Creation Page:



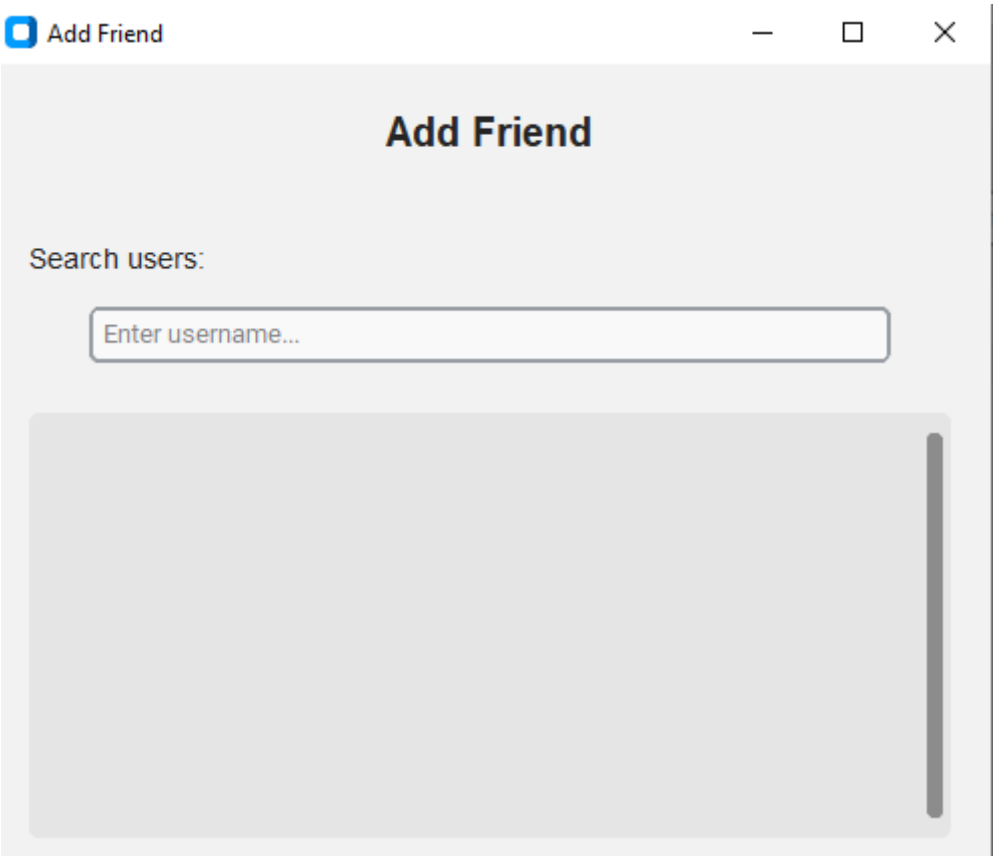
5) Like And Comment Working:



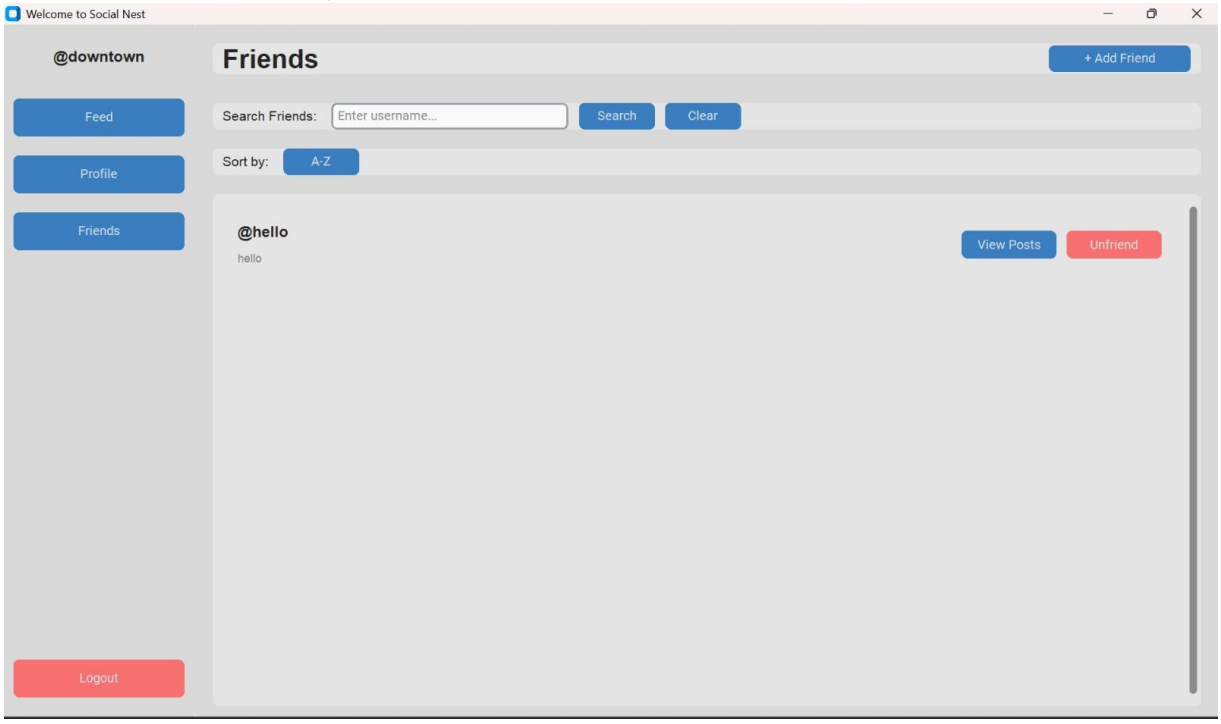
6) Comment Modal Window:



7) Make Connection Page:



8) My Connection:



Performance Analysis and Time Complexity:

Contactly delivers lightning-fast core operations thanks to intelligent data structure choices. User registration, login, post creation, liking, commenting, and friend-request actions all run in **O(1)** time using hash maps, doubly linked list head insertion, and simple queue appends. Even post deletion and undo (both navigation and deleted-post recovery) remain **O(1)** due to prev/next pointers and a classic stack.

Feed display and keyword searches require **O(n)** time as they traverse the doubly linked lists or scan content linearly, which is perfectly acceptable for typical desktop-scale datasets. Sorting friends or search results alphabetically uses Quick Sort with **O(m log m)** average time, while trending posts are sorted by likes using Insertion Sort in near **O(n)** because only a few likes change at a time. Navigation history and undo stack consume only **O(h)** extra space, where h is usually less than 20.

Overall space complexity is a clean **O(U + P + F)** — proportional to the number of users, posts, and friendships — with most writes being append-only **O(1)** operations on .jsonl files. This combination ensures real-time responsiveness for every social feature while keeping memory usage minimal and predictable. The design mirrors the efficiency of real-world platforms, making Contactly both academically excellent and practically performant.

Conclusion:

This project not only fulfills all academic objectives with flying colors but also serves as a strong foundation for a real-world application.

In the future, I plan to extend Contactly by adding real-time chat (using WebSockets), notifications with a priority queue, job postings with advanced filtering, and a recommendation engine using graph traversal algorithms — turning it into a complete career platform, InshaAllah.